Calculator using Lex and Yacc

A calculator supporting + - * /, brackets, decimal numbers and unary minus.

lexer.l

```
%{
#include <stdlib.h>
#include "y.tab.h"
%}

%%

[0-9]+(\.[0-9]+)?   { yylval.dval = atof(yytext); return NUM; }
[ \t]+              ;
\n                  { return '\n'; }
.                   { return yytext[0]; }

%%

int yywrap() { return 1; }
```

parser.y

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex();
void yyerror(const char *s);
%}

%union { double dval; }

%token <dval> NUM
%type  <dval> expr

%left '+' '-'
%left '*' '/'
%right UMINUS

%%

input : /* empty */
      | input line
      ;

line  : '\n'
      | expr '\n'       { printf("Result = %g\n", $1); }
      | error '\n'      { yyerrok; }
      ;

expr  : expr '+' expr   { $$ = $1 + $3; }
      | expr '-' expr   { $$ = $1 - $3; }
      | expr '*' expr   { $$ = $1 * $3; }
      | expr '/' expr   {
                            if ($3 == 0) { printf("Error: Division by zero\n"); YYERROR; }
                            $$ = $1 / $3;
                        }
      | '-' expr %prec UMINUS   { $$ = -$2; }
      | '(' expr ')'    { $$ = $2; }
      | NUM             { $$ = $1; }
```

```
      ;

%%

void yyerror(const char *s) {
    printf("Error: %s\n", s);
}

int main() {
    printf("Enter expressions (Ctrl+D to quit):\n");
    yyparse();
    return 0;
}
```

How to run

```
lex lexer.l
yacc -d parser.y
gcc lex.yy.c y.tab.c -o calc
./calc
```

Sample input and output

```
2 + 3 * 4       ->  Result = 14
(2 + 3) * 4     ->  Result = 20
10 / 4          ->  Result = 2.5
-5 + 8          ->  Result = 3
8 / 2 / 2       ->  Result = 2
7 / 0           ->  Error: Division by zero
2 +             ->  Error: syntax error
```